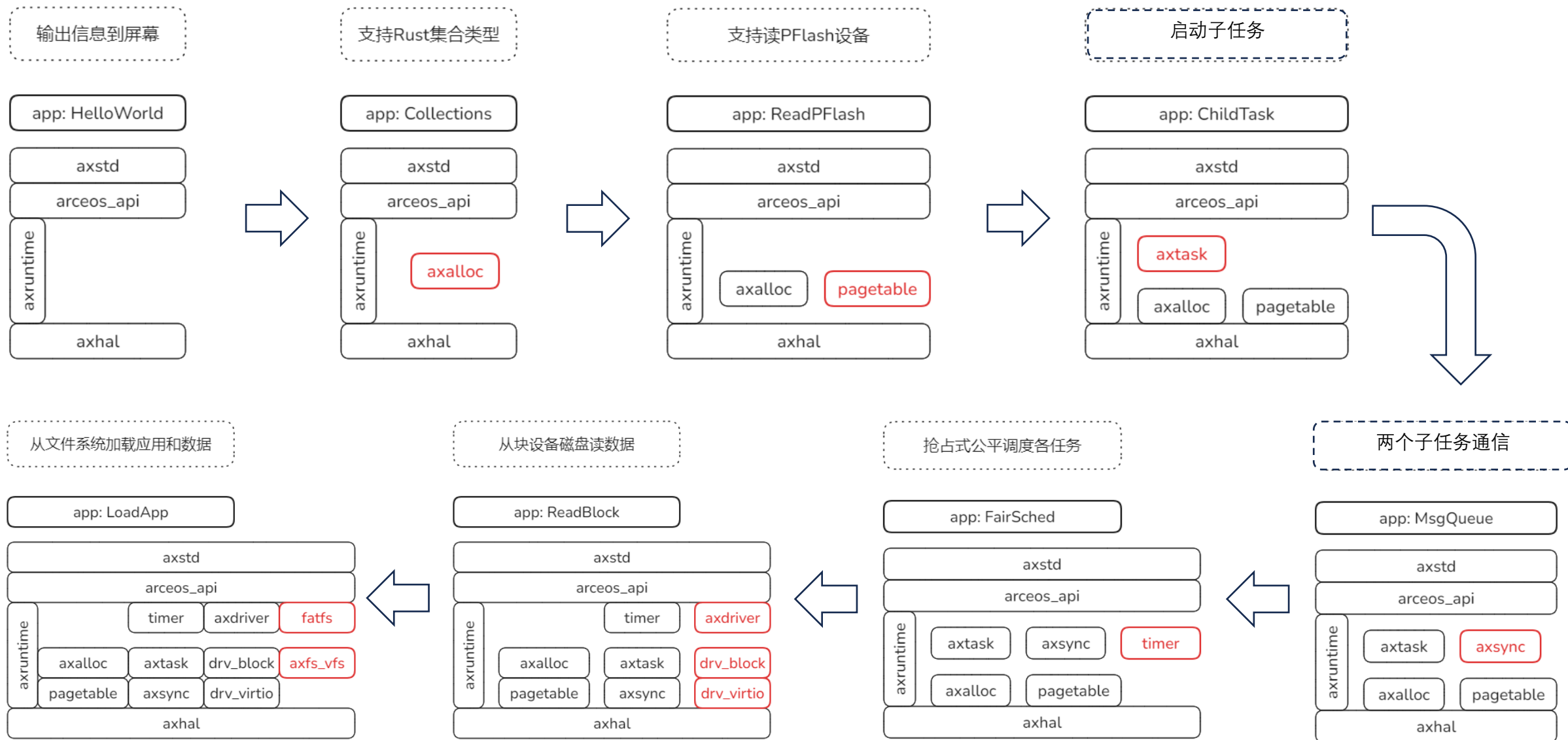


秋冬季训练营三阶段 组件化内核设计与实践(2)

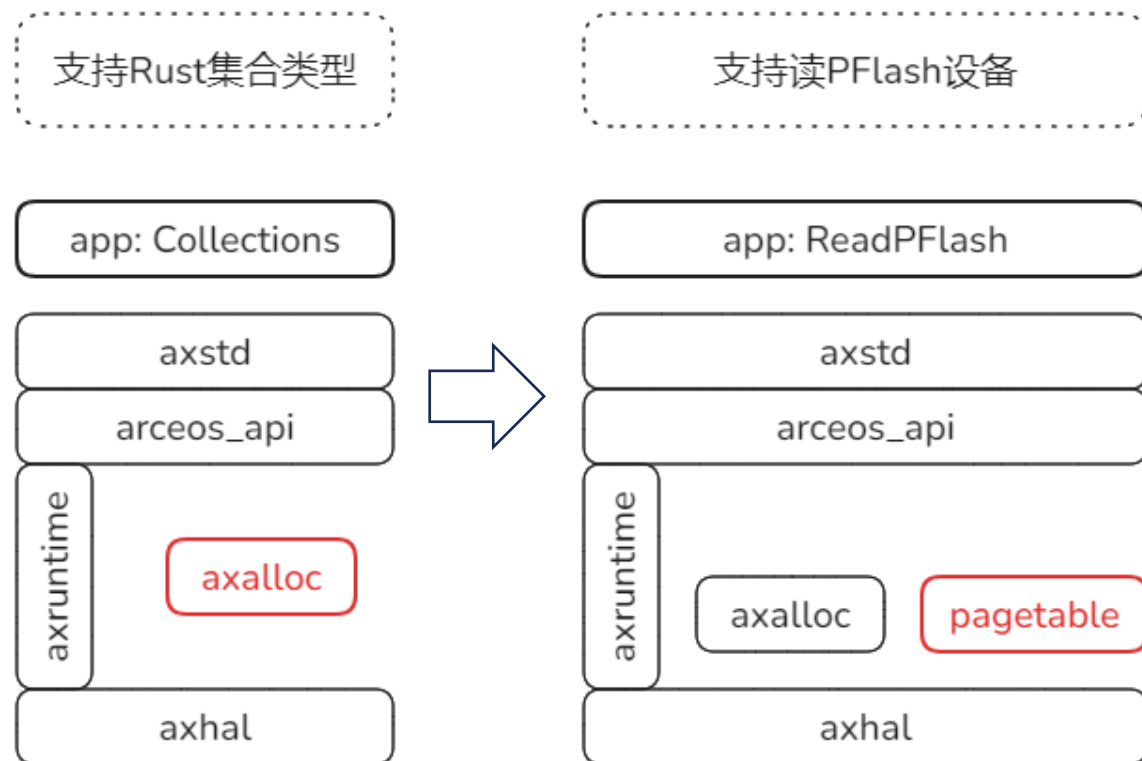
清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

2024.11.13

第一部分 Unikernel



U.3.0 ReadPFlash



```
// Makesure that we can access pflash region.
let va = phys_to_virt(PFLASH_START.into()).as_usize();
let ptr = va as *const u32;
unsafe {
    println!("Try to access dev region [{:#X}], got {:#X}",
             va, *ptr);
    let magic = mem::transmute::<u32, [u8; 4]>(*ptr);
    println!("Got pflash magic: {}",
             str::from_utf8(&magic).unwrap());
}
```

```
[dependencies]
axstd = { features = ["alloc", "paging"],
```

实验命令行: make run A=tour/u_3_0
(尝试没有指定"paging"时的情况)

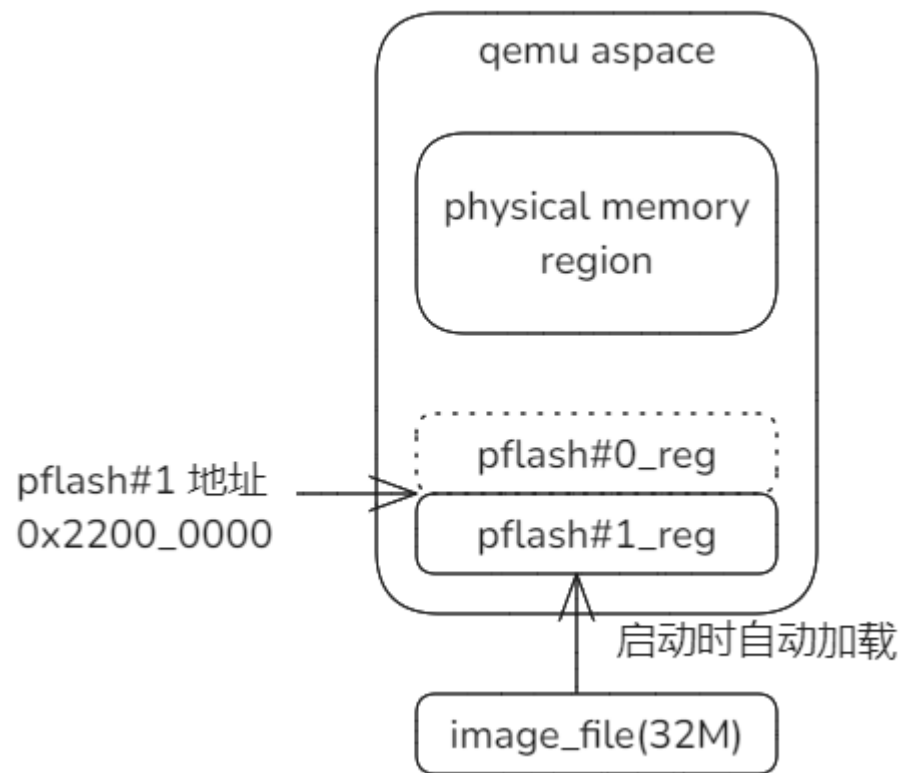
本节目标:

1. 引入页表管理组件, 通过地址空间重映射, 支持设备MMIO
2. 地址空间概念, 重映射的意义, 页表机制

需要解决的问题

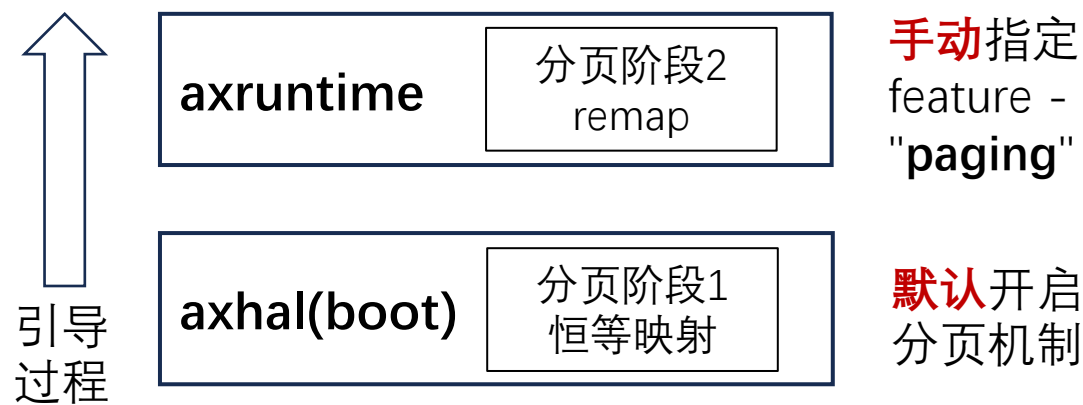
1. PFlash的作用？

Qemu的PFlash模拟闪存磁盘，启动时自动从文件加载内容到固定的MMIO区域，而且对读操作不需要驱动，可以直接访问。

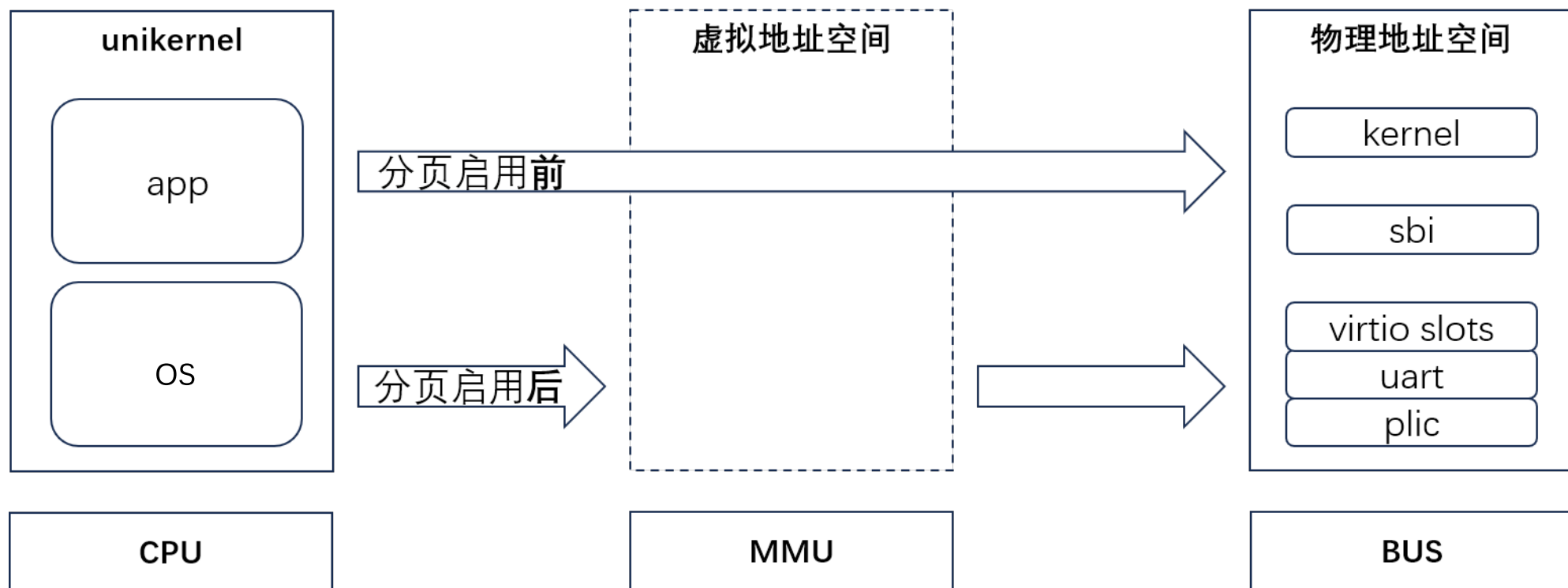


2. 为何不指定"paging"时导致读PFlash失败？

ArceOS Unikernel包括两阶段地址空间映射，Boot阶段默认开启1G空间的恒等映射；如果需要在设备MMIO区间，通过指定一个feature - "paging"来实现重映射。



地址空间与启用



物理地址空间

以riscv64-qemu为例:

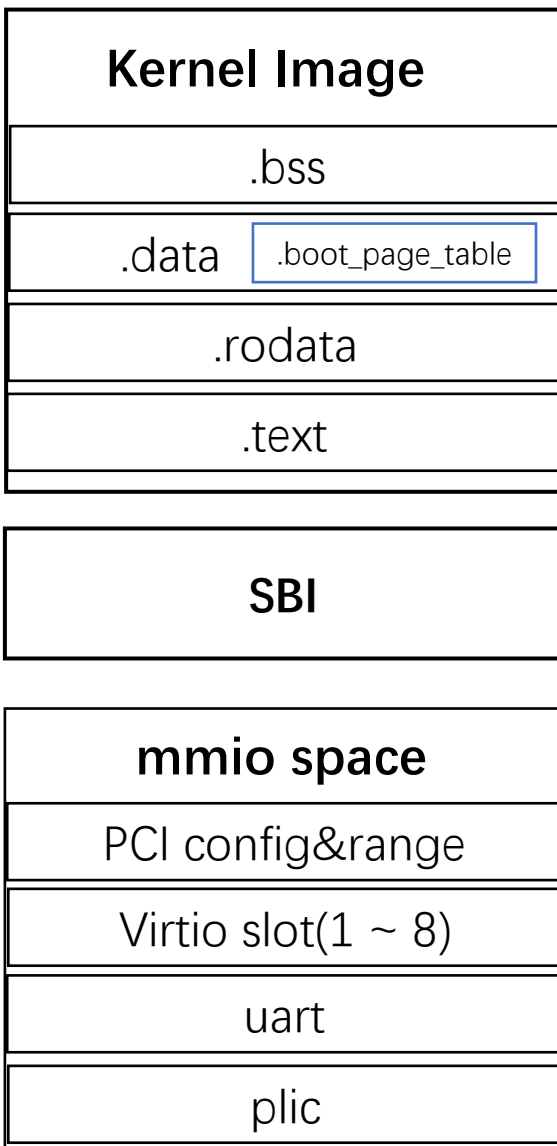
modules/axhal

linker_riscv64-qemu-virt.lds:
指导rust的链接器组织段布局

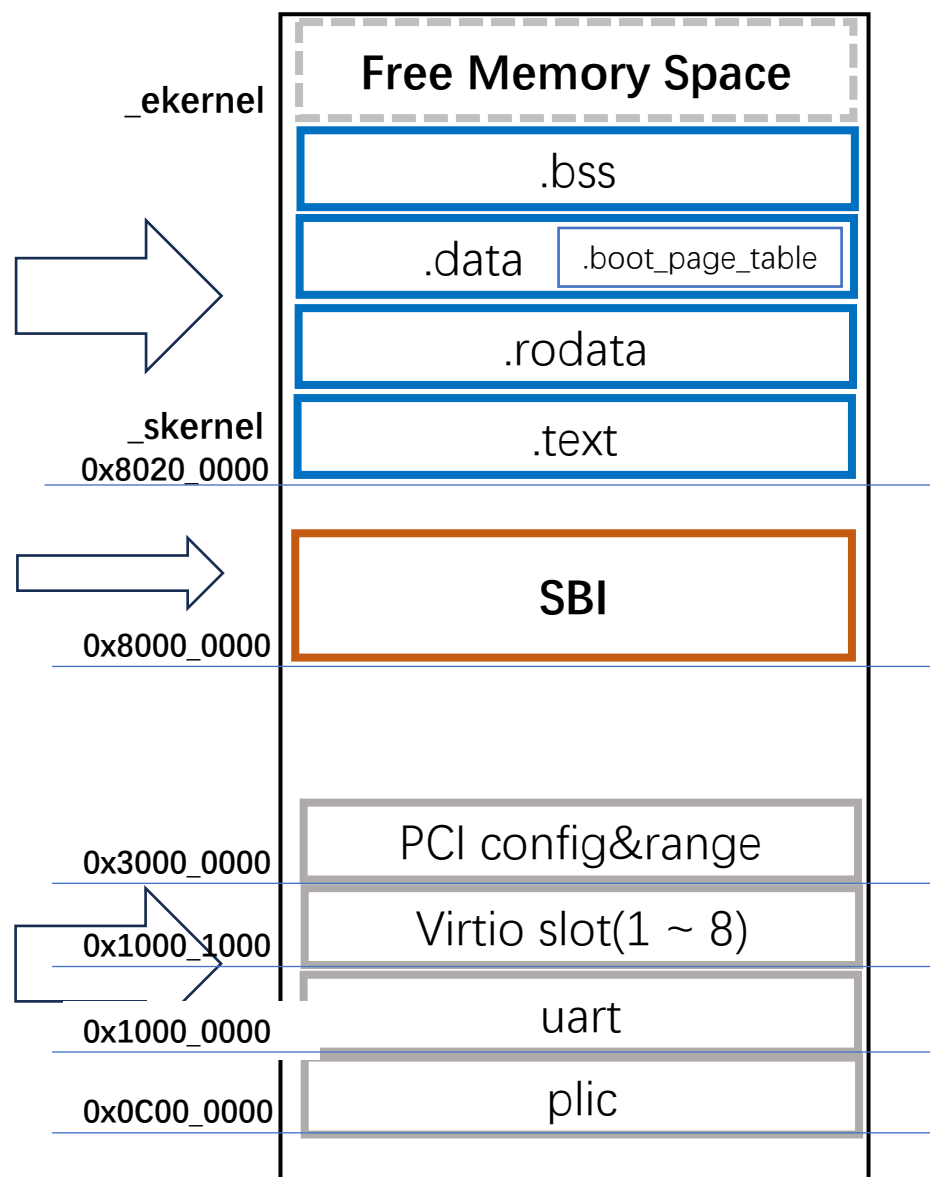
SBI配置**FW_TEXT_START**:
BIOS负责把SBI加载到内存的
起始位置

qemu fdt:
规定各个设备地址空间范围

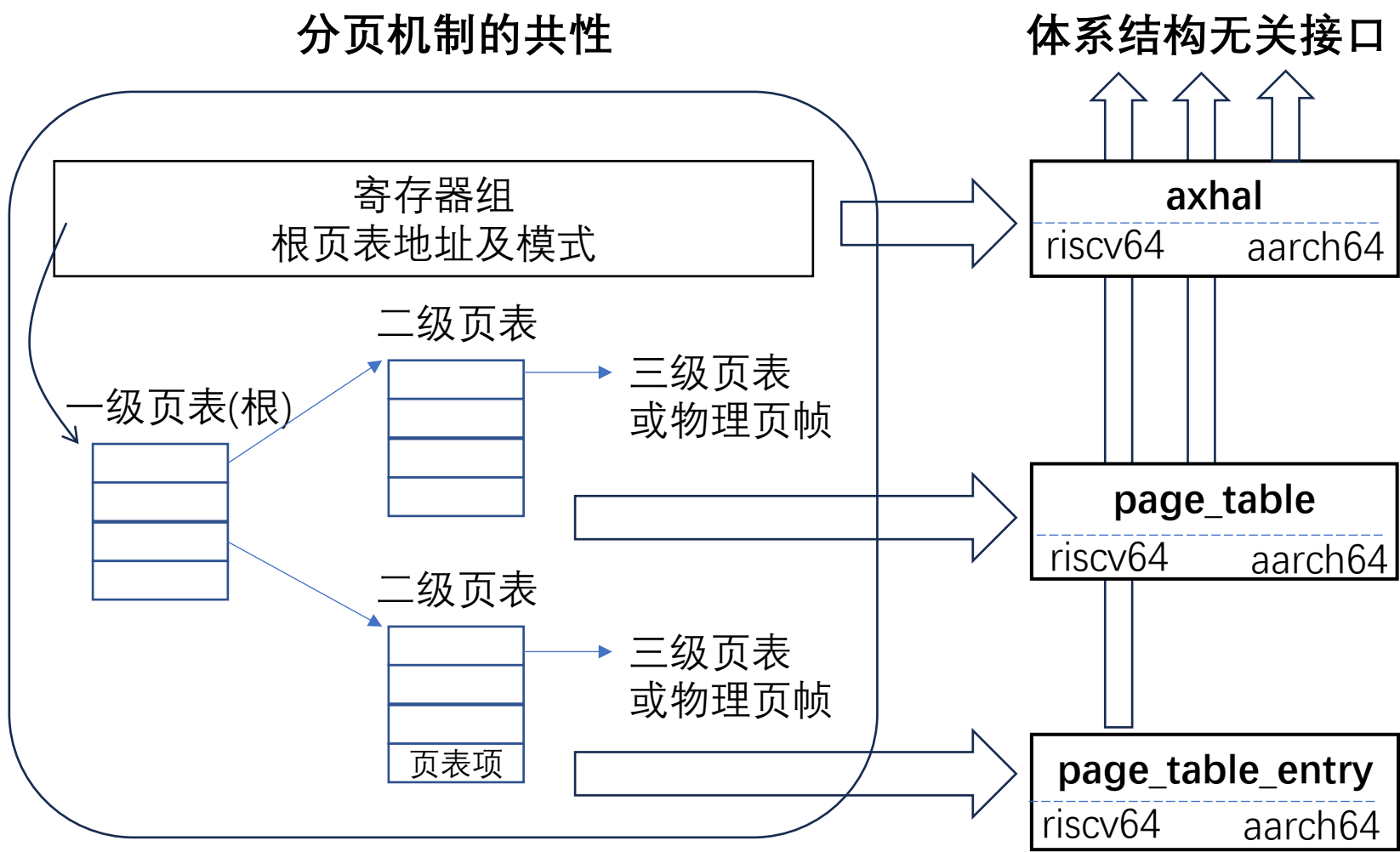
静态定义



(运行时)物理地址空间



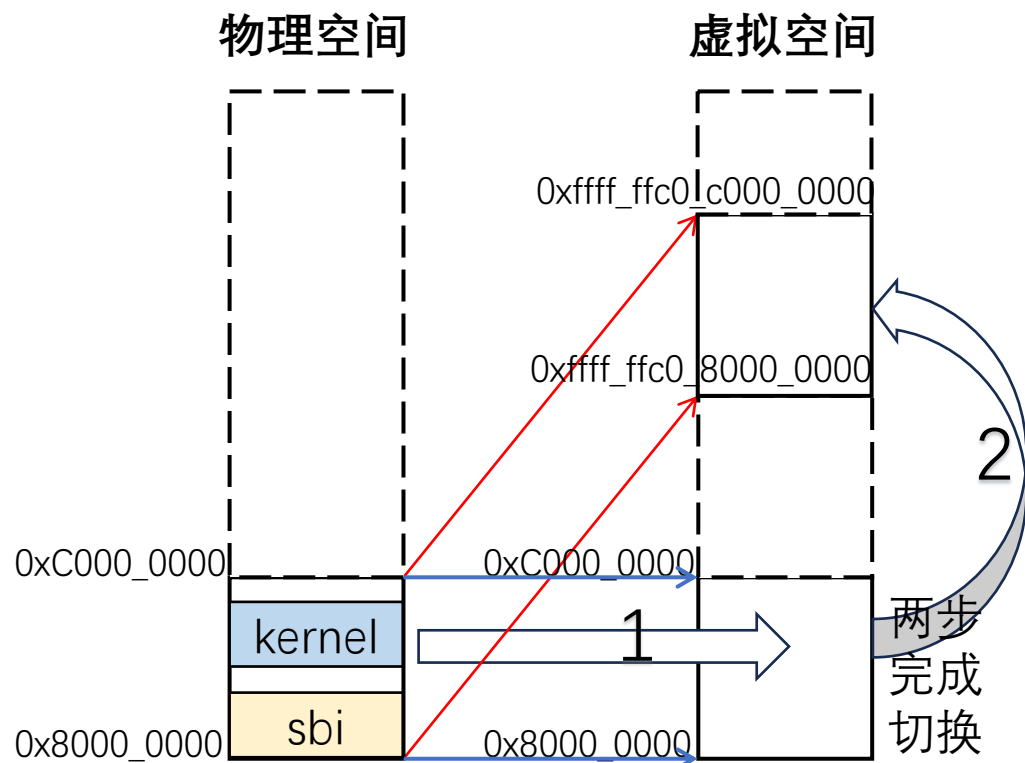
分页 - 功能抽象和对应组件



分页阶段1 – 早期启用(必须)

分页启用的两个阶段：早期启用(必须)和后期重建映射(可选)。

阶段1： 内核启动的早期，采用规定的恒等映射方式。但是只映射一部分物理空间。



以Riscv64的实现为例

目标： 完成Paging切换后，建立从虚拟空间0xffff_ffc0_8000_0000 ~ 0xffff_ffc0_8000_0000到物理空间0x8000_0000~0xC000_0000 的映射，范围1G。

两步完成Paging切换：

1. 恒等映射保证虚拟空间与物理空间有一个相等范围的地址空间映射(0x80000000~0xC0000000)。切换前后地址范围不变，但地址空间已经从物理空间切换到虚拟空间。
2. 给指令指针寄存器pc，栈寄存器sp等加偏移，在图中该偏移是0xffff_ffc0_0000_0000。如此在虚拟空间执行平移后，就完成到最终目标地址的映射。

分页阶段1 – 早期启用(必须) – 代码示例

```
modules/axhal/src/platform/riscv64_qemu_virt/boot.rs

#[link_section = ".data.boot_page_table"]
static mut BOOT_PT_SV39: [u64; 512] = [0; 512];

unsafe extern "C" fn _start() -> ! {
    // PC = 0x8020_0000
    // a0 = hartid
    // a1 = dtb
    core::arch::asm!(
        mv      s0, a0           // save hartid
        mv      s1, a1           // save DTB pointer
        la      sp, {boot_stack}
        li      t0, {boot_stack_size}
        add     sp, sp, t0       // setup boot stack

        call    {init_boot_page_table}
        call    {init_mmu}      // setup boot page table and enable MMU
    );
}

unsafe fn init_boot_page_table() {
    // 0x8000_0000..0xc000_0000, VRWX_GAD, 1G block
    BOOT_PT_SV39[2] = (0x80000 << 10) | 0xef;
    // 0xffff_ffc0_8000_0000..0xffff_ffc0_c000_0000, VRWX_GAD, 1G block
    BOOT_PT_SV39[0x102] = (0x80000 << 10) | 0xef;
}

unsafe fn init_mmu() {
    let page_table_root = BOOT_PT_SV39.as_ptr() as usize;
    satp::set(satp::Mode::Sv39, 0, page_table_root >> 12);
    riscv::asm::sfence_vma_all();
}
```

BOOT_PT_SV39使用的是LDS定义布局时，直接预留的一页，所以不用额外内存分配。

	[53:10]物理页帧	[9:0]属性
0		
1		
2	0x80000	DAG_XWRV
...		
0x102	0x80000	DAG_XWRV

初始化根页表BOOT_PT_SV39，只有一级，即每个页表项直接映射到1G的地址空间。
1G = 2³⁰ 因此pgd_idx = (VA >> 30) & (512-1)
0x8000_0000 >> 30，对应pgd_idx = 2
0xffff_ffc0_8000_0000 >> 30，只保留低9位，对应pgd_idx = 0x102

物理页帧号 = 物理地址 >> 12，故0x80000

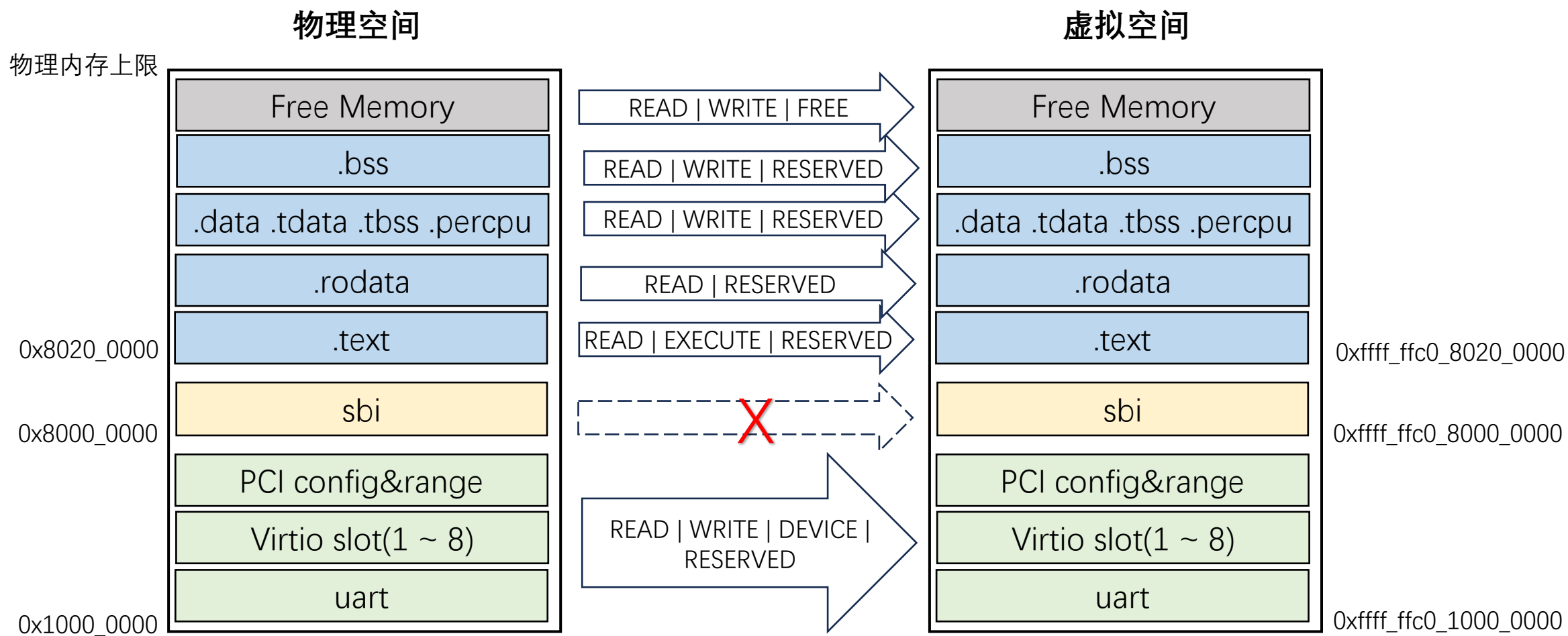
分页阶段2 – 重建映射(可选)

阶段2:指定paging feature的情况下，启动后期重建完整的空间映射。

注: paging不是决定分页是否启用，而是决定是否包含阶段2。

重建映射的意义:

1. 管理更大范围的地址空间，包括设备的MMIO范围
2. 分类和权限的细粒度控制



分页阶段2 – 重建映射-示例

modules/axruntime/src/lib.rs

```
#[cfg(feature = "paging")]  
axmm::init_memory_management();
```

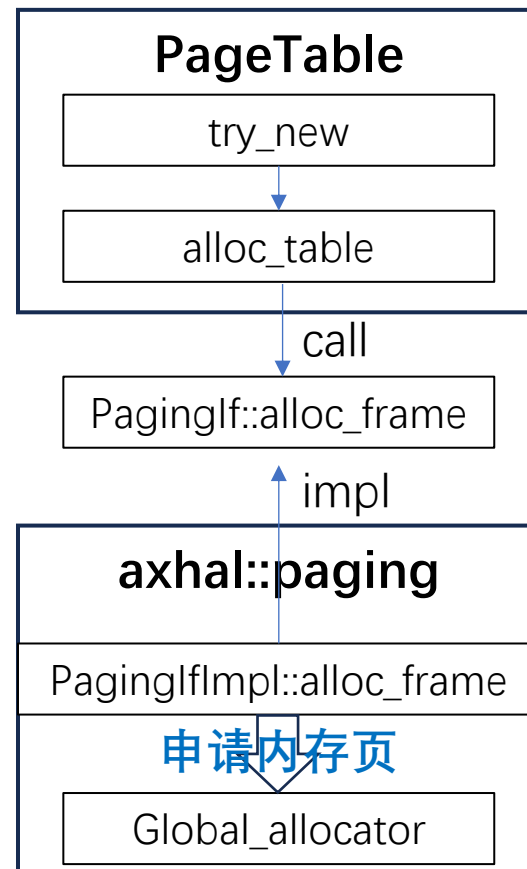
modules/axmm/src/lib.rs

```
pub fn init_memory_management() {  
    info!("Initialize virtual memory management...");  
  
    let kernel_aspace = new_kernel_aspace().expect("failed to initialize  
    debug!("kernel address space init OK: {:#x?}", kernel_aspace);  
    KERNEL_ASPACE.init_once(SpinNoIrq::new(kernel_aspace));  
    axhal::paging::set_kernel_page_table_root(kernel_page_table_root());  
}  
  
pub fn new_kernel_aspace() -> AxResult<AddrSpace> {  
    let mut aspace = AddrSpace::new_empty(  
        va!(axconfig::KERNEL_ASPACE_BASE),  
        axconfig::KERNEL_ASPACE_SIZE,  
    )?;  
}
```

modules/axmm/src/aspace.rs

```
pub(crate) fn new_empty(base: VirtAddr, size: usize) -> AxResult<Self> {  
    Ok(Self {  
        va_range: VirtAddrRange::from_start_size(base, size),  
        areas: MemorySet::new(),  
        pt: PageTable::try_new().map_err(|_| AxError::NoMemory)?,  
    })  
}
```

axhal提供体系结构无关的接口方法
set_kernel_page_table
写入根页表地址并启用分页



小结

内存管理框架与功能：

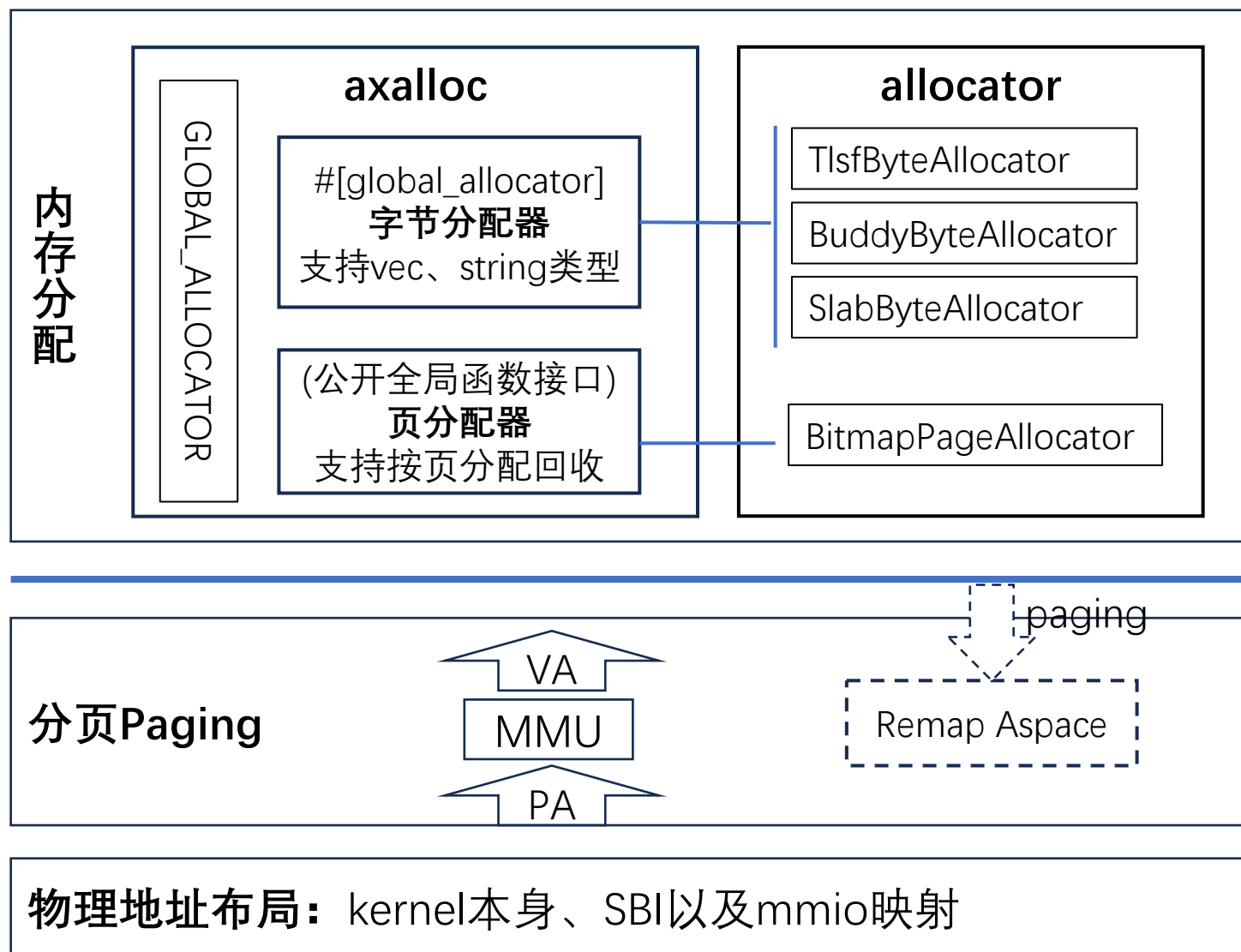
1) 内存分配功能

内含两类分配器，字节分配器和页分配器。

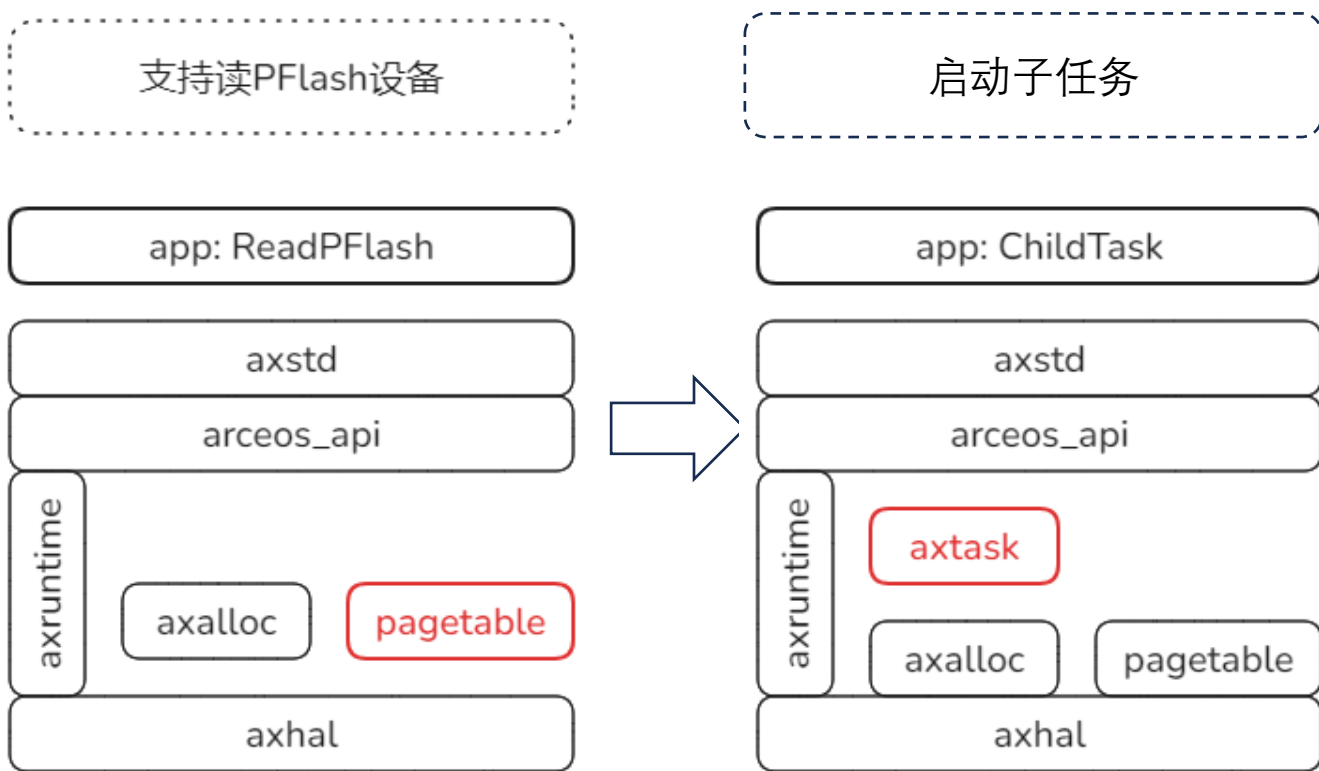
框架与算法分离，松耦合支持多种内存分配算法。

2) 分页功能

启动早期基于静态恒等映射完成分页切换，如果指定paging feature则会在启动后期重新建立范围更大，权限控制更细的映射。



U.4.0 ChildTask



```
let worker = thread::spawn(move || {
    println!("Spawned-thread ...");

    // Makesure that we can access pflash region.
    let va = phys_to_virt(PFLASH_START.into()).as_usize()
    let ptr = va as *const u32;
    let magic = unsafe {
        mem::transmute::<u32, [u8; 4]>(*ptr)
    };
    if let Ok(s) = str::from_utf8(&magic) {
        println!("Got pflash magic: {s}");
        0
    } else {
        -1
    }
});
```

```
[dependencies]
axstd = { features = ["alloc", "paging", "multitask"],
```

实验命令行: `make run A=tour/u_4_0`

本节目标:

1. 启动第一个子任务, 建立多任务的概念和基本框架
2. 框架要素: 任务、运行队列

需要解决的问题

1. 对于单CPU，多任务并发的基本原理？

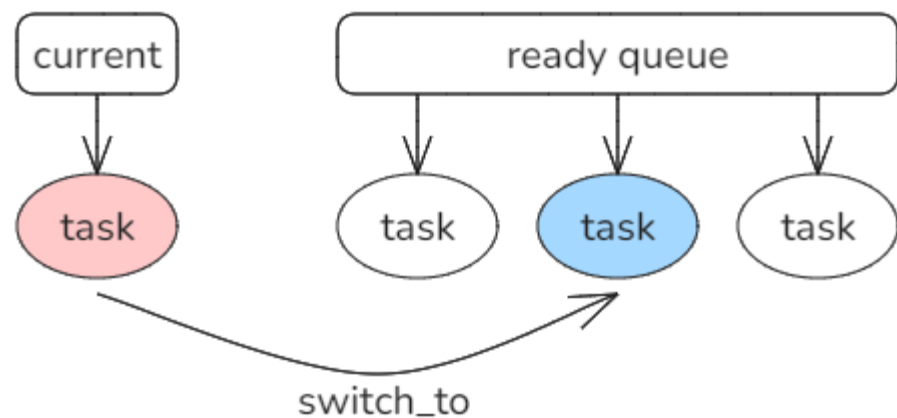
多个任务(执行上下文)分时复用CPU的时间资源。



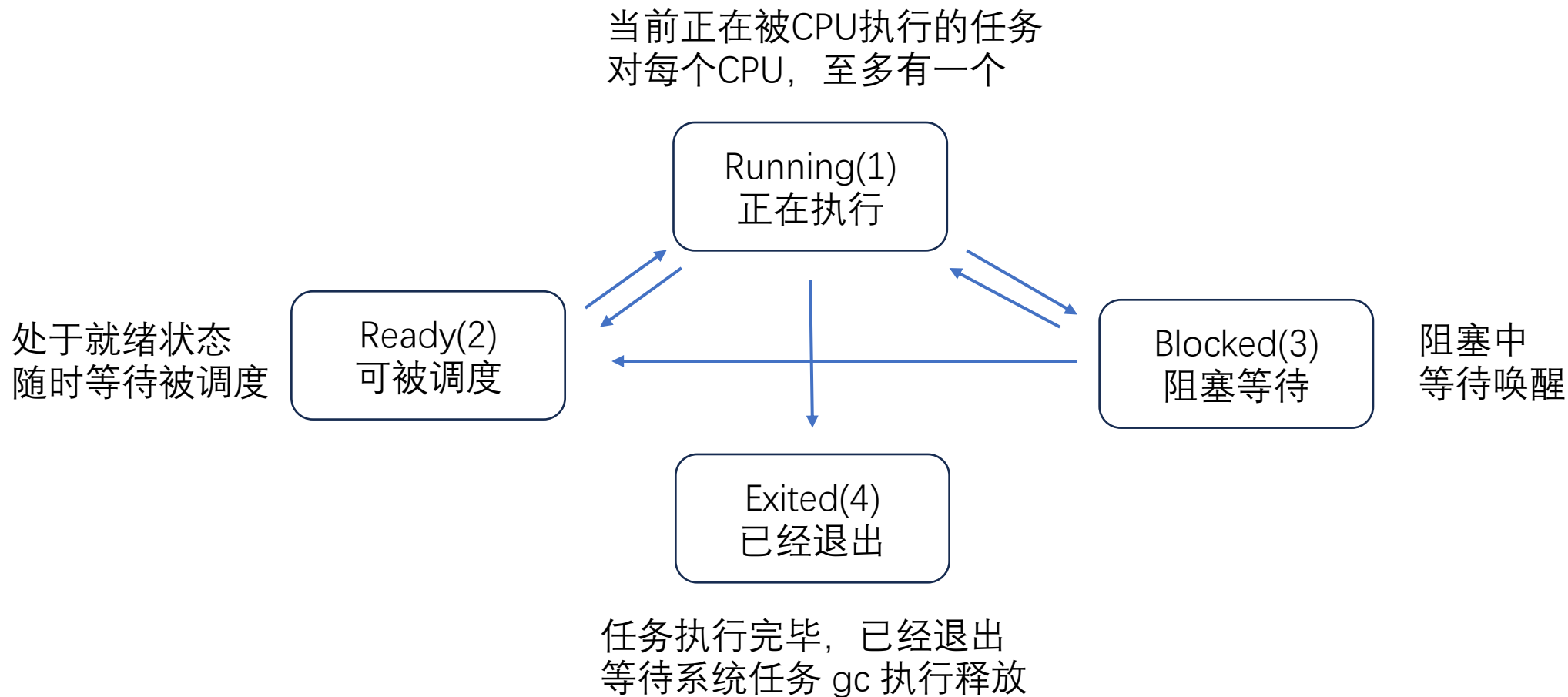
- 对单CPU，每个时刻只能运行一个任务，是**并发**而非**并行**
- 各个任务分享时间资源的比例由**调度策略**决定
- 在当前任务被调出时，能够完整保存它在此时的执行状态，即**保存现场**；确保下次被重新调入时能够**完整恢复现场**，任务能够无感知的从上次断点处继续执行

2. 任务调度的基本机制？

CPU当前正在执行任务与就绪队列中的某个任务进行对换。



任务与任务状态



任务数据结构

```
pub struct TaskInner {  
    id: TaskId,  
    name: String,  
    is_idle: bool,  
    is_init: bool,  
  
    entry: Option<*mut dyn FnOnce()>,  
    state: AtomicU8,  
  
    in_wait_queue: AtomicBool,  
    #[cfg(feature = "irq")]  
    in_timer_list: AtomicBool,  
  
    #[cfg(feature = "preempt")]  
    need_resched: AtomicBool,  
    #[cfg(feature = "preempt")]  
    preempt_disable_count: AtomicUsize,  
  
    exit_code: AtomicI32,  
    wait_for_exit: WaitQueue,  
  
    kstack: Option<TaskStack>,  
    ctx: UnsafeCell<TaskContext>,  
    task_ext: AxTaskExt,  
  
    #[cfg(feature = "tls")]  
    tls: TlsArea,  
}
```

is_idle: 本身是系统任务idle
is_init: 本身是任务main(其实就是主线程)

entry: 实现任务逻辑函数的入口
state: 任务状态, 即上页所示的四个状态

kstack: ArceOS任务相当于线程, 所以具有自己的栈空间

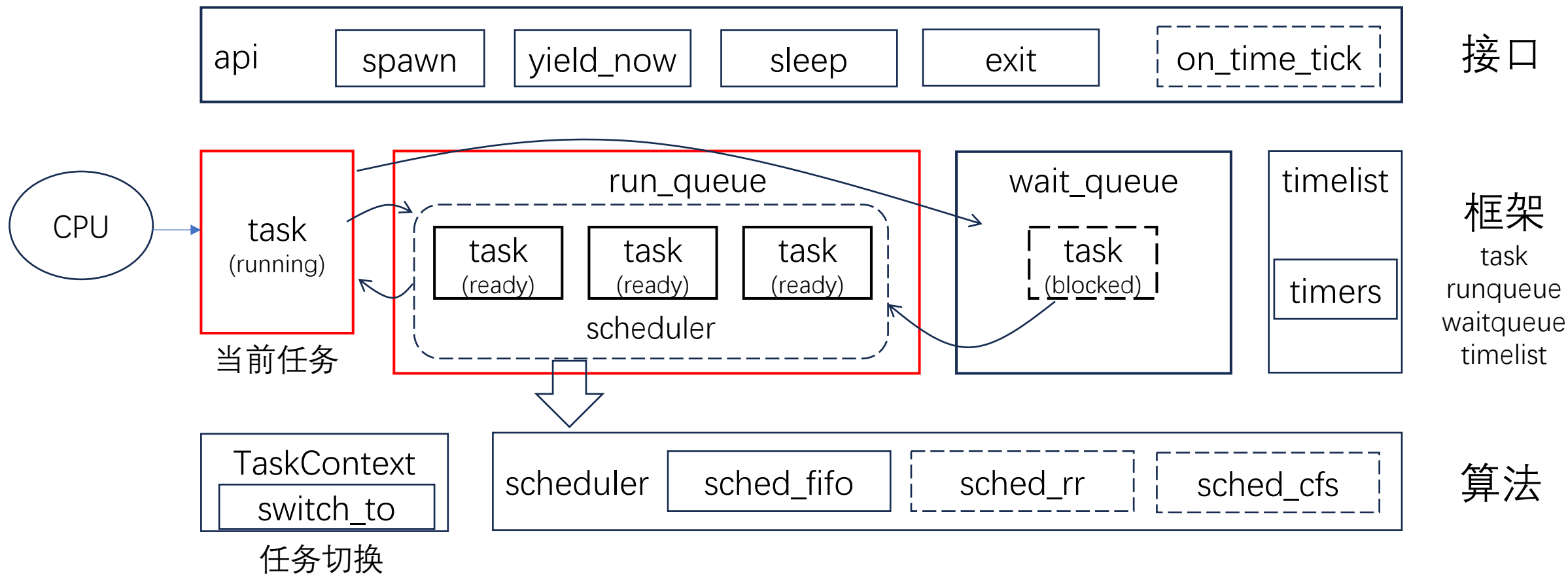
ctx: 上下文类型TaskContext,
调度的核心数据结构
保存恢复任务的关键

任务的扩展属性, 是面向宏内核和Hypervisor扩展的关键;
但对于Unikernel, 它是空, 本节略过。

通用调度框架

支持协作式和抢占式调度的通用框架。

标红的两处：当前任务和任务运行队列是框架的关键。



接口 - 主要调度API

接口公开的是runqueue的对应方法

1) spawn&spawn_raw

产生一个新任务，加入runqueue，处于Ready

2) yield_now (协作式调度的关键)

主动让出CPU执行权

3) sleep&sleep_until

睡眠固定的时间后醒来

在timers定时器列表中注册，等待唤醒

4) exit

当前任务退出，标记状态，等待GC回收

```
pub fn spawn_raw<F>(f: F, name: String, stack_size: usize) -> AxTaskRef
where
    F: FnOnce() + Send + 'static,
{
    let task = TaskInner::new(f, name, stack_size);
    RUN_QUEUE.lock().add_task(task.clone());
    task
}

pub fn add_task(&mut self, task: AxTaskRef) {
    debug!("task spawn: {}", task.id_name());
    assert!(task.is_ready());
    self.scheduler.add_task(task);
}
```

```
pub fn yield_current(&mut self) {
    let curr = crate::current();
    trace!("task yield: {}", curr.id_name());
    assert!(curr.is_running());
    self.resched(false);
}

fn resched(&mut self, preempt: bool) {
    let prev = crate::current();
    if prev.is_running() {
        prev.set_state(TaskState::Ready);
        if !prev.is_idle() {
            self.scheduler.put_prev_task(prev.clone(), preempt);
        }
    }
    let next = self.scheduler.pick_next_task().unwrap_or_else(|| unsafe {
        // Safety: IRQs must be disabled at this time.
        IDLE_TASK.current_ref_raw().get_unchecked().clone()
    });
    self.switch_to(prev, next);
}
```

调度框架初始化

modules/axruntime/src/lib.rs

```
#[cfg(feature = "multitask")]  
axtask::init_scheduler();
```

axruntime: 当指定multitask feature时, 初始化框架

modules/axtask/src/api.rs

```
/// Initializes the task scheduler (for the primary CPU).  
pub fn init_scheduler() {  
    info!("Initialize scheduling...");  
  
    crate::run_queue::init();  
    #[cfg(feature = "irq")]  
    crate::timers::init();  
  
    info!(" use {} scheduler.", Scheduler::scheduler_name());  
}
```

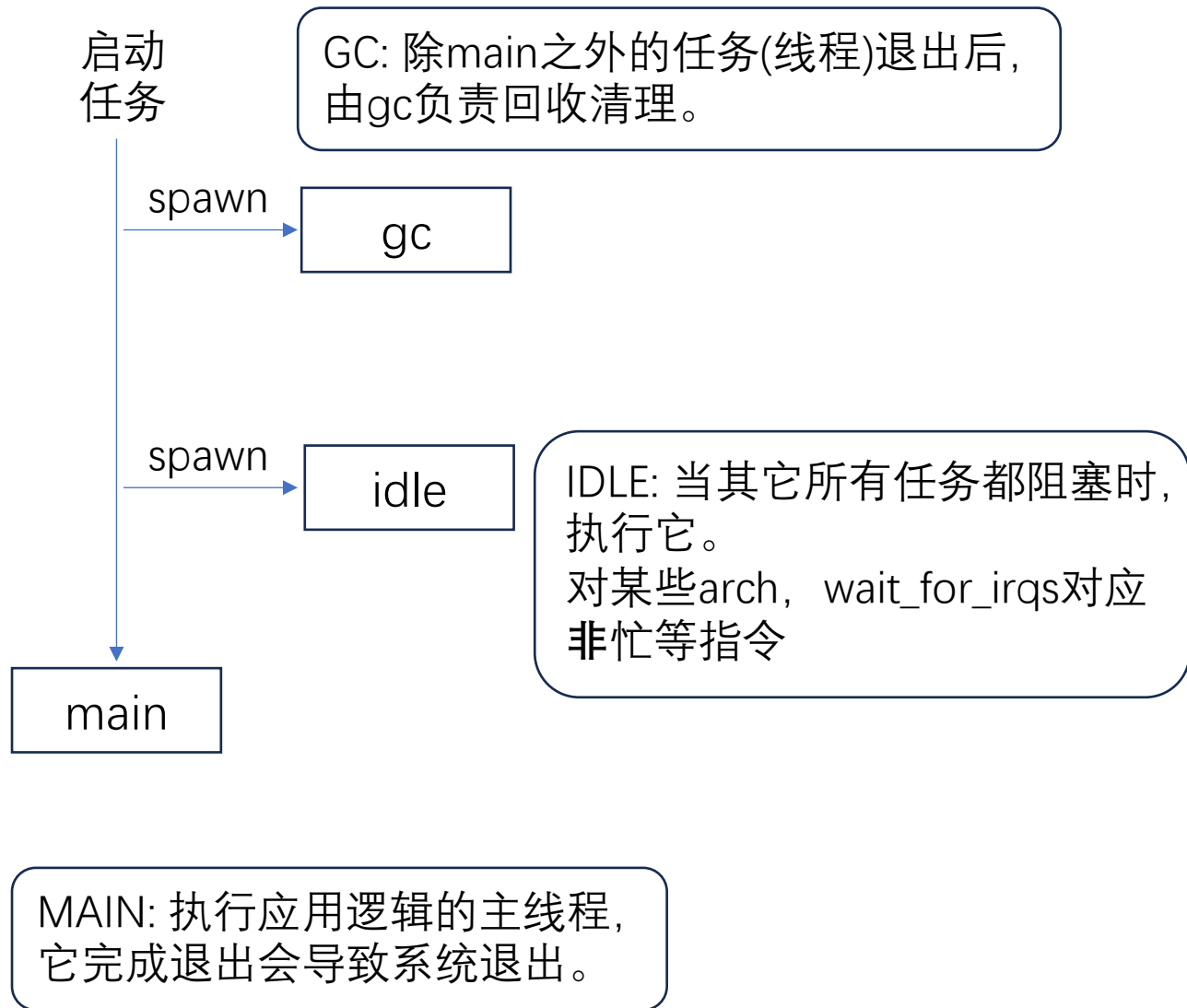
run_queue是任务调度框架的核心
timers负责维护定时器列表, 支持sleep等API的实现

modules/axtask/src/run_queue.rs

```
pub(crate) fn init() {  
    const IDLE_TASK_STACK_SIZE: usize = 4096;  
    let idle_task = TaskInner::new(|| crate::run_idle(), "idle".into(), IDLE_TASK_STACK_SIZE);  
    IDLE_TASK.with_current(|i| i.init_by(idle_task.clone()));  
  
    let main_task = TaskInner::new_init("main".into());  
    main_task.set_state(TaskState::Running);  
  
    RUN_QUEUE.init_by(AxRunQueue::new());  
    unsafe { CurrentTask::init_current(main_task) }  
}
```

初始化系统任务: idle main gc

系统默认内置任务



```
pub fn exit_current(&mut self, exit_code: i32) -> ! {
    let curr = crate::current();
    debug!("task exit: {}, exit_code={}", curr.id_name(), exit_code);
    assert!(curr.is_running());
    assert!(!curr.is_idle());
    if curr.is_init() {
        EXITED_TASKS.lock().clear();
        axhal::misc::terminate();
    } else {
        curr.set_state(TaskState::Exited);
        curr.notify_exit(exit_code, self);
        EXITED_TASKS.lock().push_back(curr.clone());
        WAIT_FOR_EXIT.notify_one_locked(false, self);
        self.resched(false);
    }
    unreachable!("task exited!");
}
```

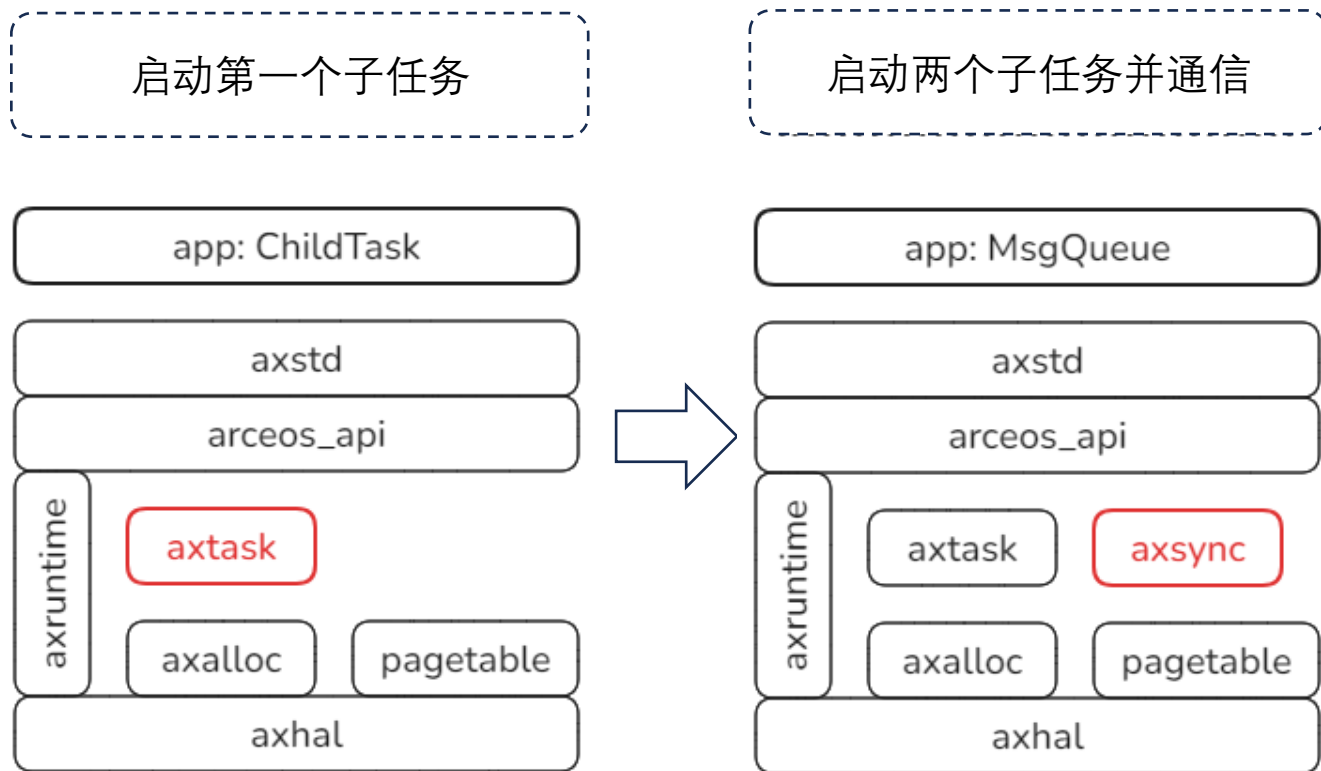
```
pub fn run_idle() -> ! {
    loop {
        yield_now();
        debug!("idle task: waiting for IRQs...");
        #[cfg(feature = "irq")]
        axhal::arch::wait_for_irqs();
    }
}
```

小结

本节我们通过创建和启动第一个线程任务，建立了任务调度的基本框架，了解了调度相关的关键要素：任务和任务运行队列。

下节将启动两个任务，在它们之间建立通信。
进一步学习调度的核心机制 - 任务上下文切换 以及 协作式调度算法。

U.5.0 MsgQueue



本节目标:

1. 任务的切换机制, 协作式调度算法
2. 同步的方式, Mutex的机制

```
let worker1 = thread::spawn(move || {
    println!("worker1 ...");
    for i in 0..=LOOP_NUM {
        println!("worker1 [{i}]");
        q1.lock().push_back(i);
        // NOTE: If worker1 doesn't yield, others have
        // no chance to run until it exits!
        thread::yield_now();
    }
    println!("worker1 ok!");
});

let worker2 = thread::spawn(move || {
    println!("worker2 ...");
    loop {
        if let Some(num) = q2.lock().pop_front() {
            println!("worker2 [{num}]");
            if num == LOOP_NUM {
                break;
            }
        } else {
            println!("worker2: nothing to do!");
            // TODO: it should sleep and wait for notify!
            thread::yield_now();
        }
    }
    println!("worker2 ok!");
});
```

实验命令行: make run A=tour/u_5_0

需要解决的问题

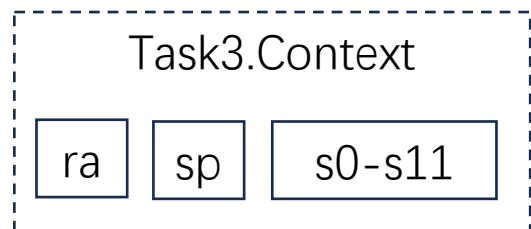
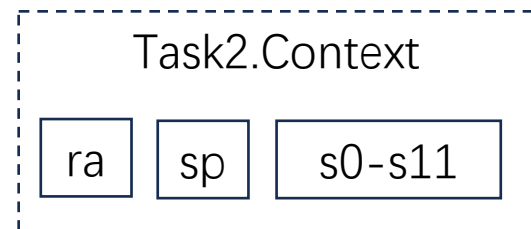
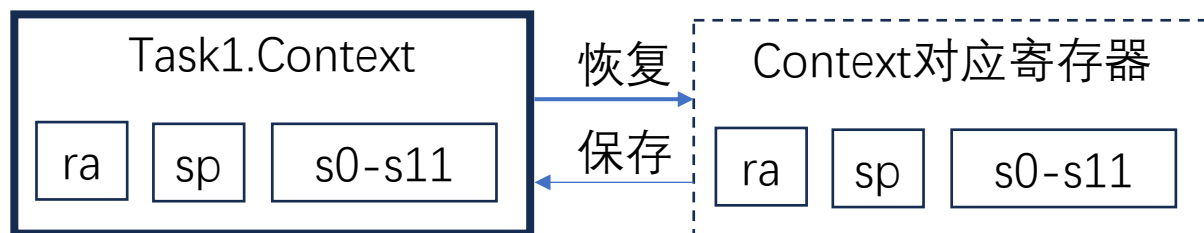
任务间切换如何实现？

任务上下文Context: 保存任务状态的最小的寄存器状态集合。

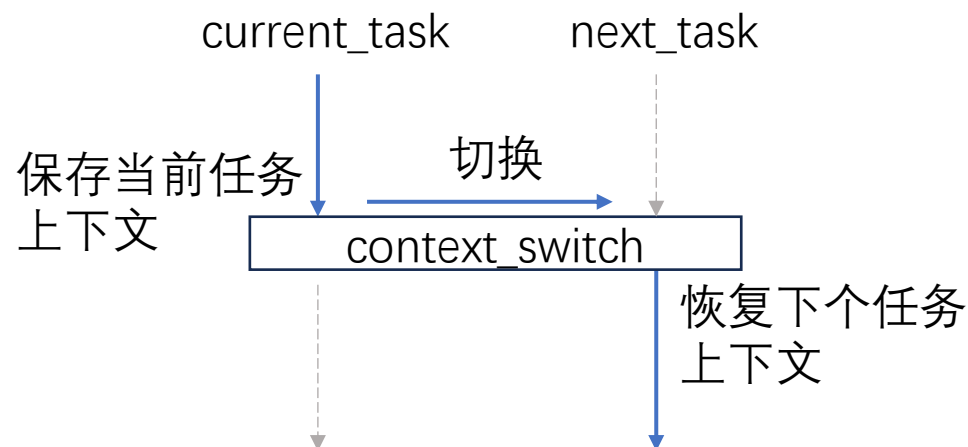
下面是理解上下文切换的两个角度：

内存中各任务的Context存储

CPU实际寄存器组

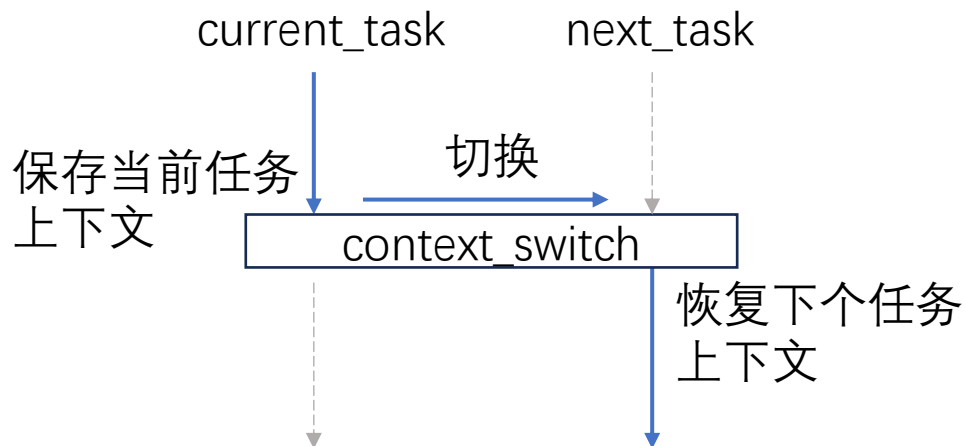


物理上每个CPU只存在一套寄存器，其中部分寄存器与任务状态直接相关，它们决定当前是哪个任务在运行。任务切入就是对应保存的上次寄存器状态载入；切出时，保存以备下次再次载入。



任务切换涉及两个任务，通过特殊函数 context_switch 完成。特殊之处：某任务作为函数调用者进入函数后，状态保存后被挂起；函数返回后，执行权被交给另一个任务。

核心算法： context_switch



上下文Context包含寄存器:

- 1) ra: 函数返回地址寄存器，这个切换实现了任务执行指令流的切换。
- 2) sp: 任务即线程，这个是线程栈
- 3) s0~s11: 按照riscv规范，callee不能改这组寄存器的信息，所以需要保存。

```
#[naked]
unsafe extern "C" fn context_switch(
    _current_task: &mut TaskContext, _next_task: &TaskContext
) {
    asm!(
        // save old context (callee-saved registers)
        STR    ra, a0, 0
        STR    sp, a0, 1
        STR    s0, a0, 2
        STR    s1, a0, 3
        STR    s2, a0, 4
        STR    s3, a0, 5
        STR    s4, a0, 6
        STR    s5, a0, 7
        STR    s6, a0, 8
        STR    s7, a0, 9
        STR    s8, a0, 10
        STR    s9, a0, 11
        STR    s10, a0, 12
        STR    s11, a0, 13

        // restore new context
        LDR    s11, a1, 13
        LDR    s10, a1, 12
        LDR    s9, a1, 11
        LDR    s8, a1, 10
        LDR    s7, a1, 9
        LDR    s6, a1, 8
        LDR    s5, a1, 7
        LDR    s4, a1, 6
        LDR    s3, a1, 5
        LDR    s2, a1, 4
        LDR    s1, a1, 3
        LDR    s0, a1, 2
        LDR    sp, a1, 1
        LDR    ra, a1, 0

        ret",
        options(noreturn),
    )
}
```

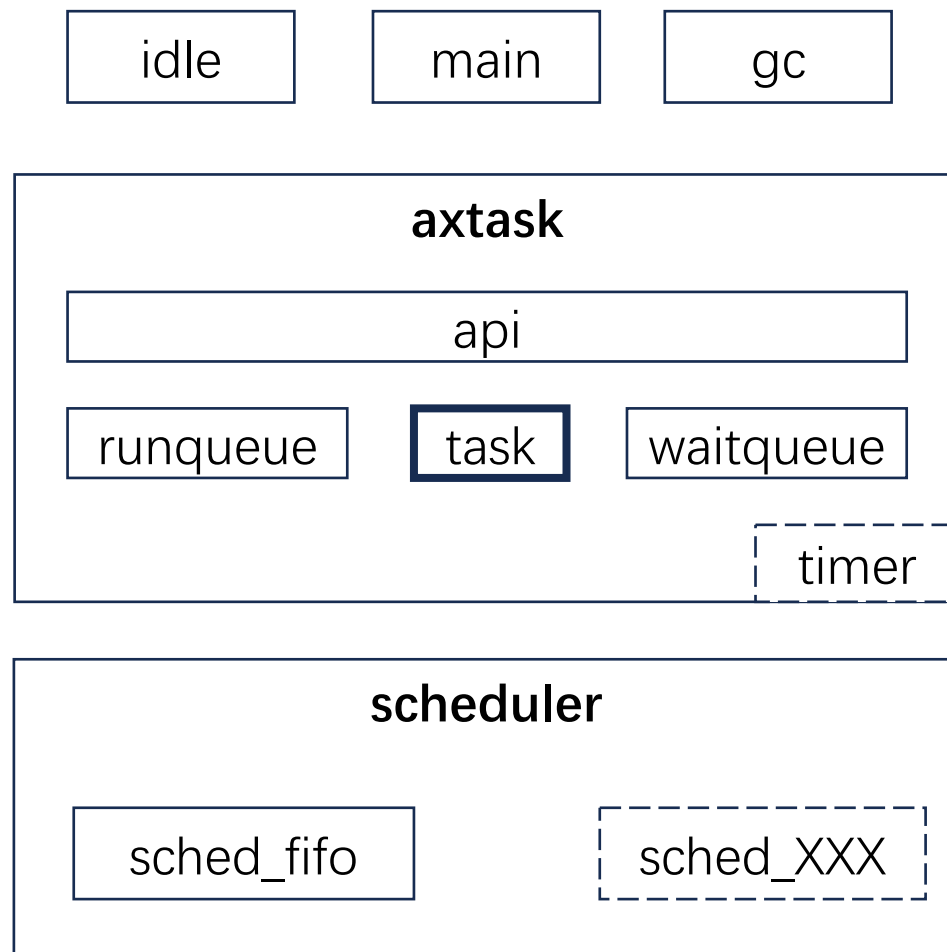
调度与协作式调度

任务：被调度的对象，具有独立工作逻辑。

调度：资源相对于使用者通常是不足的，调度就是用来协调每个请求对资源的使用的方法。

任务调度：协调可执行任务对 CPU 资源 的使用。

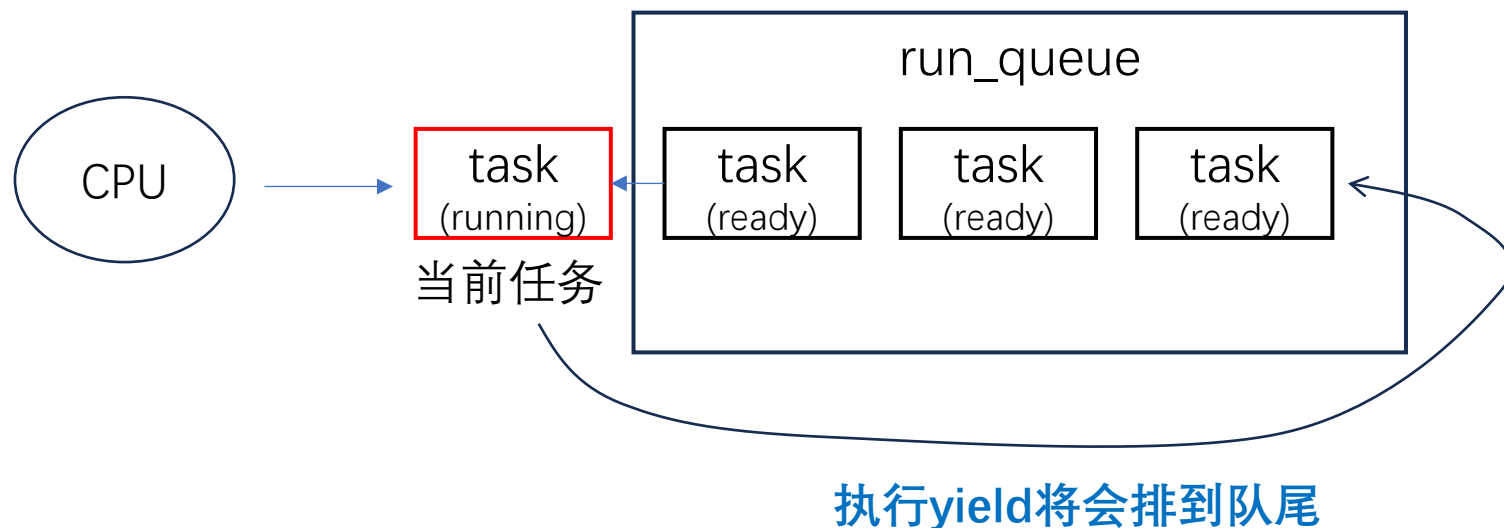
协作式调度：任务之间通过“友好”协作方式分享CPU资源。具体的，当前任务是否让出和何时让出CPU控制权完全由当前任务自己决定。



协作式调度算法FIFO机制

协作式FIFO机制：

- 1) 任务按照先入先出的顺序被CPU执行
- 2) 当前任务一旦获得执行权，就会一直执行。
- 3) 只有两种情况下，其它任务才能获得执行机会：
 - 3.1) 当前任务执行完成后退出
 - 3.2) 当前任务主动调用yield_now让出执行权



协作式调度算法FIFO实现

```
pub struct FifoScheduler<T> {  
    ready_queue: List<Arc<FifoTask<T>>>,  
}  
impl<T> BaseScheduler for FifoScheduler<T> {  
    type SchedItem = Arc<FifoTask<T>>;  
  
    fn init(&mut self) {}  
  
    fn add_task(&mut self, task: Self::SchedItem) {  
        self.ready_queue.push_back(task);  
    }  
  
    fn remove_task(&mut self, task: &Self::SchedItem) -> Option<Self::SchedItem> {  
        unsafe { self.ready_queue.remove(task) }  
    }  
  
    fn pick_next_task(&mut self) -> Option<Self::SchedItem> {  
        self.ready_queue.pop_front()  
    }  
  
    fn put_prev_task(&mut self, prev: Self::SchedItem, _preempt: bool) {  
        self.ready_queue.push_back(prev);  
    }  
  
    fn task_tick(&mut self, _current: &Self::SchedItem) -> bool {  
        false // no reschedule  
    }  
  
    fn set_priority(&mut self, _task: &Self::SchedItem, _prio: isize) -> bool {  
        false  
    }  
}
```

先入先出队列，直接使用最简单的List维护任务

新任务投入运行，直接push队尾

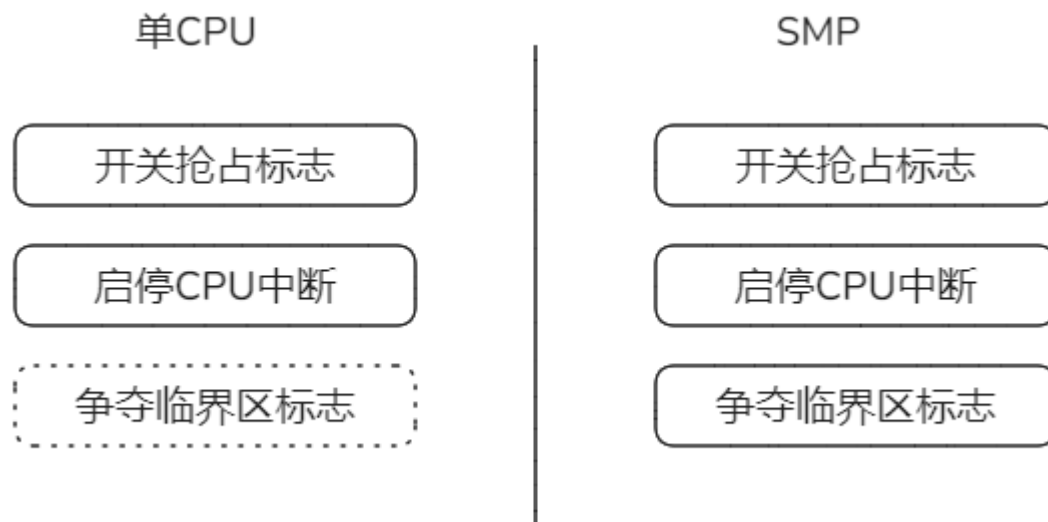
取出下一个，pop队首

放回前一个，push队尾

不支持时钟定时触发的重调度
不支持设置优先级

同步原语 - 自旋锁

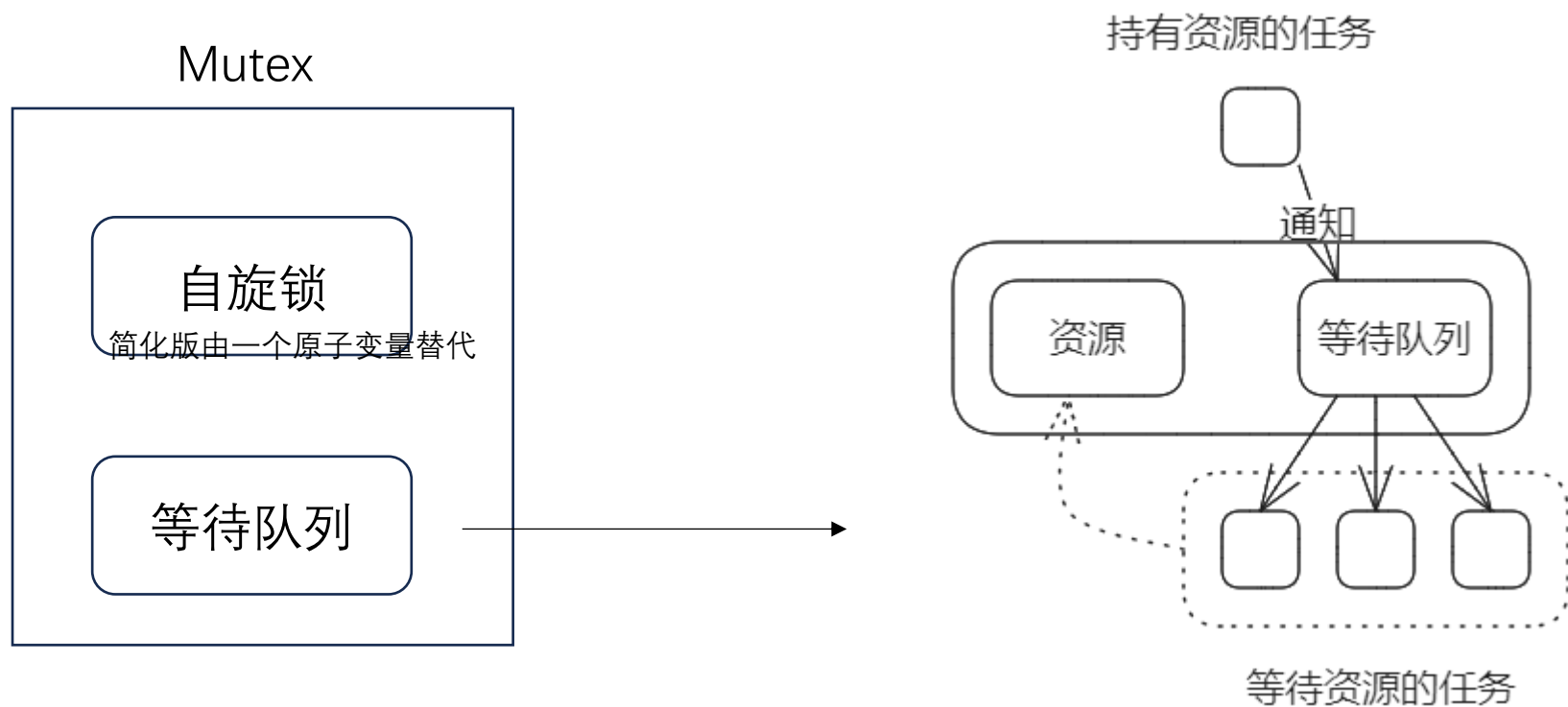
对于单CPU，加锁时只需要关中断 + 关抢占。无须额外的临界区互斥操作。
对于SMP，才需要基于相互可见的内存变量进行原子互斥操作。



同步原语 - 互斥锁(睡眠)

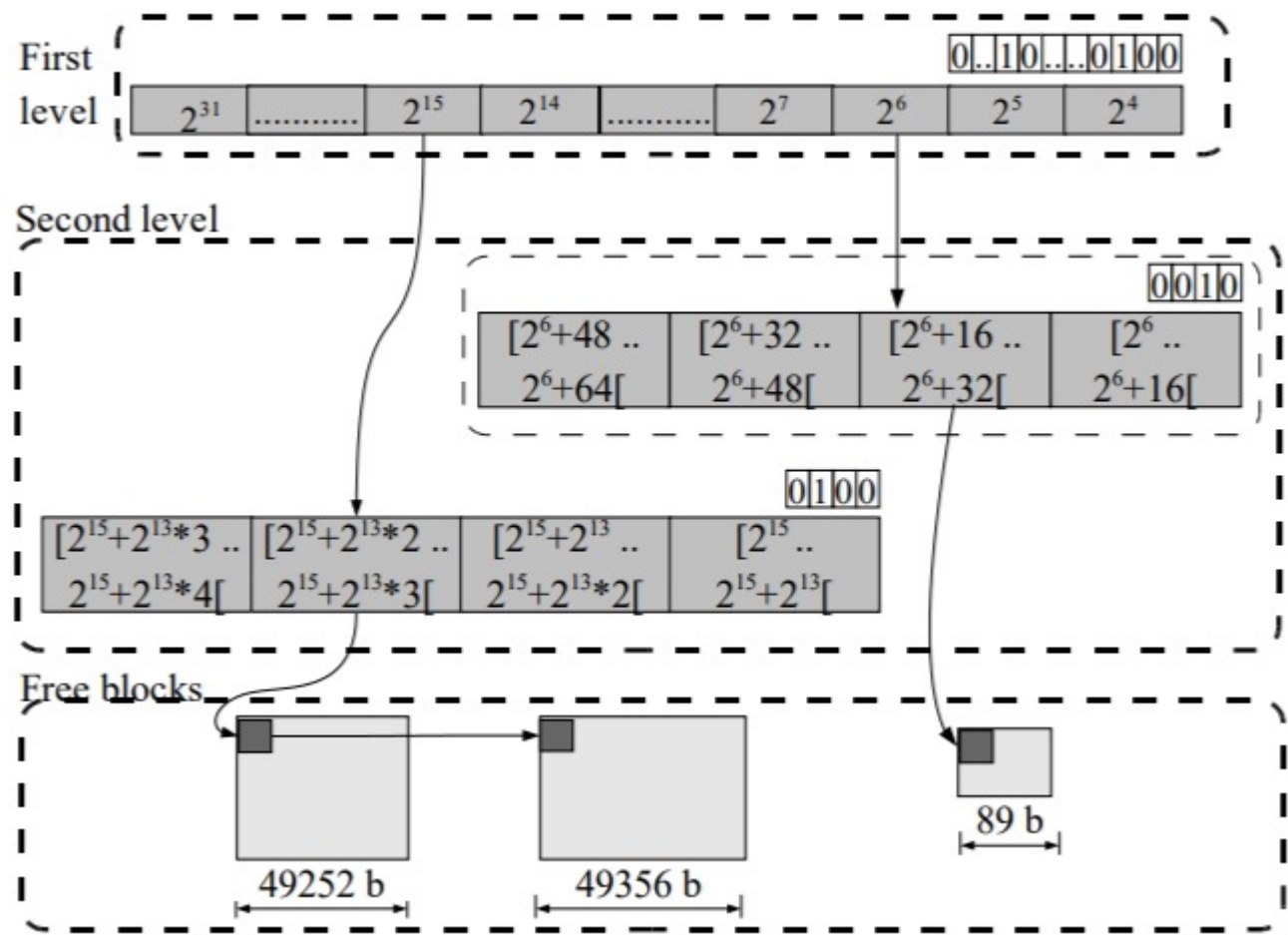
互斥锁通常可以认为是：等待队列waitq + 自旋锁spinlock。

等待队列是针对某种资源，任务之间进行协调。至多只能有一个任务持有资源，多于一个的任务进入睡眠状态，转入等待队列；直至被唤醒。



上节遗留关于内存分配算法的
部分和课后习题

内存分配算法-TLSF (Two-Level Segregated Fit)



两级bitmap+List管理空闲块

bitmap第一级First Level:
每一位对应一个范围的内存块，示例中分别对应 $2^4 \sim 2^{31}$ 。1表示空闲。图中两个1。

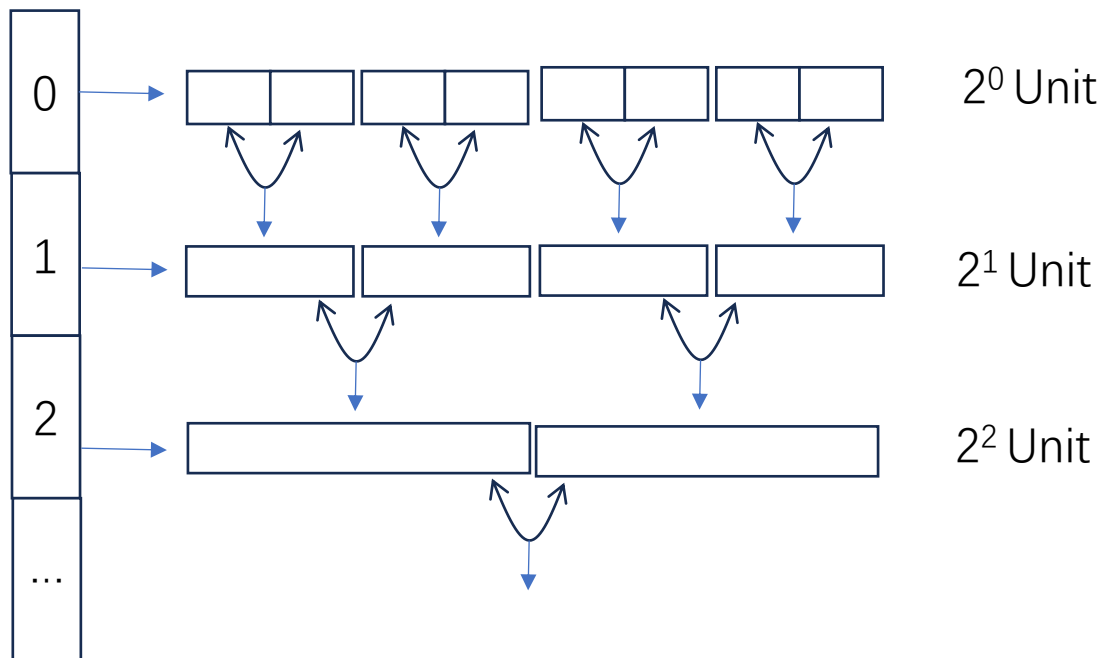
bitmap第二级Second Level:
有几位就表示几等分。例如， 2^6 表示64~127，然后进行4等分就是64~79, 80~95, 96~107, 108~127，每一位对应一个范围，同样1表示空闲。

然后就能找到包含对应范围大小的空闲块链表List。链表耗尽或者新建时，对应维护两级bitmap。

算法论文的连接: [ecrts04_tlsf.pdf \(upv.es\)](http://ecrts04_tlsf.pdf(upv.es))

内存分配算法-Buddy

Order List



分配单元Unit:

一般不会采用1字节，通常8，16，32…字节

分配:

寻找匹配alloc需要(order)的最小块

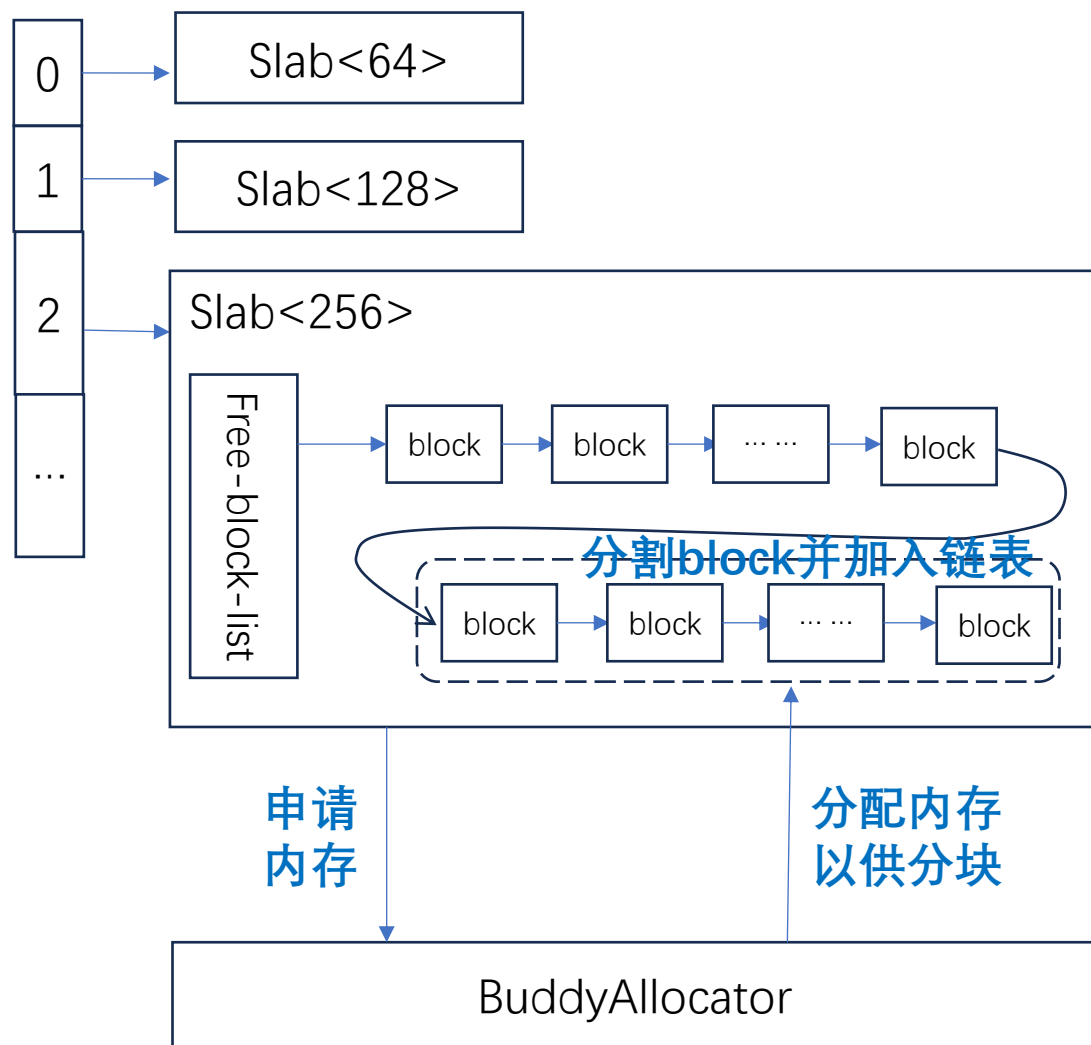
如果order大于目标，则二分切割，直至相等，每级剩余的部分挂到对应的Order List

释放:

查看是否有邻居空闲块，有则尽可能向高Order合并，直至无法合并，挂到OrderList。

内存分配算法-Slab

Order List



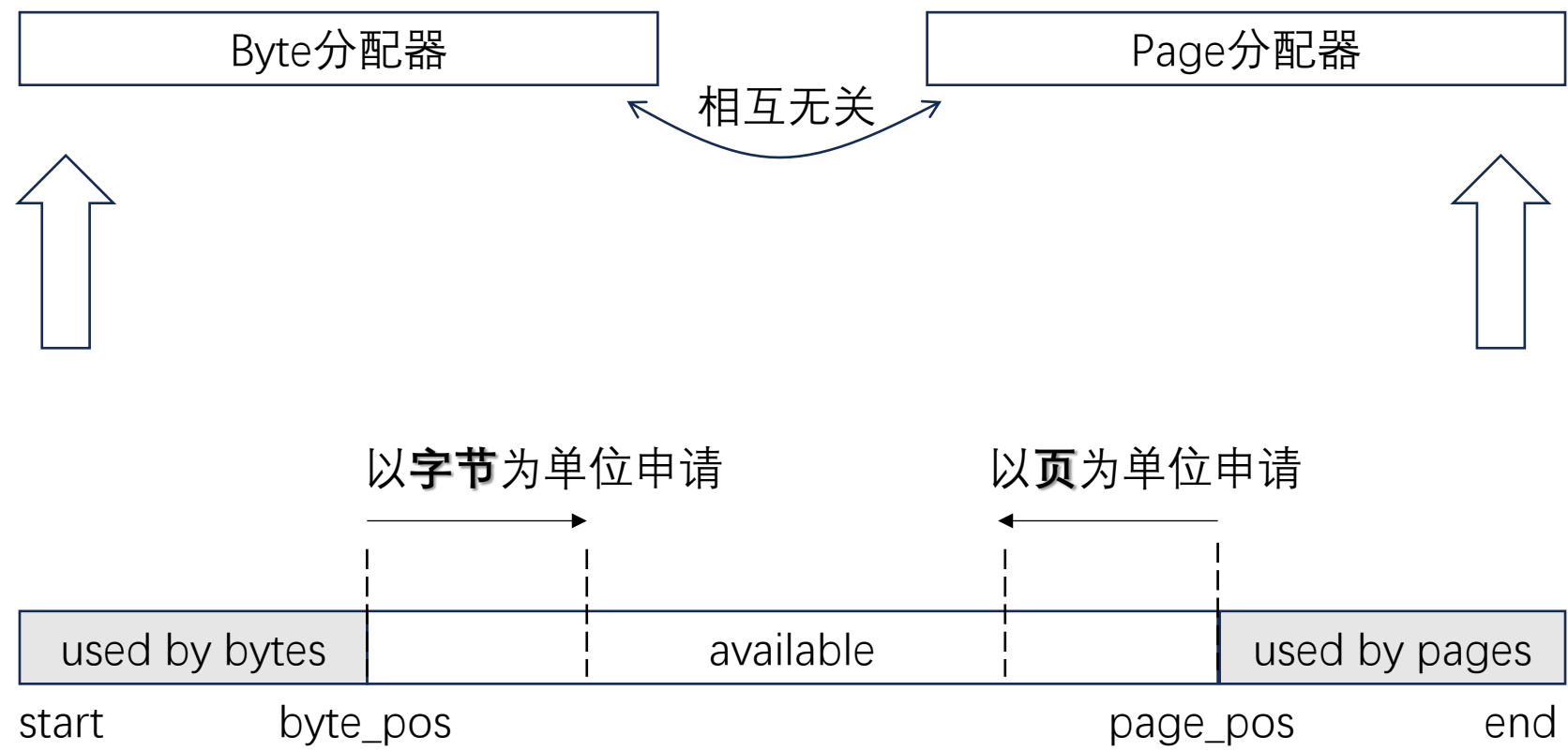
结构:

- 1) 通过OrderList维护一系列Slab
- 2) Slab维持一个空闲的block链表

分配: 从block空闲链表中弹出一个block。
依靠BuddyAllocator提供内存分配支持, 初始时以及block不足时, 从BuddyAllocator申请, 分割block后加入block空闲链表。

释放: 放回block空闲链表。

课后练习 - bump内存分配算法



课后练习 - bump内存分配算法

[alt_alloc]: 为内存分配器实现新的内存算法bump。

预备:

1. 如果没有**exercises/alt_alloc**，回到main分支执行**git pull** 更新工程，再新建分支做如下练习
2. 执行**make A=exercises/alt_alloc/ run**

```
error[E0046]: not all trait items implemented, missing: `init`, `add_memory`  
--> modules/bump_allocator/src/lib.rs:24:1  
24 | impl BaseAllocator for EarlyAllocator {  
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ missing `init`, `add_memory` in implementation
```

要求:

1. **只能**修改modules/bump_allocator组件的实现，支持bump内存分配算法。**不能**改其它部分。
2. 再次执行**make A=exercises/alt_alloc/ run**

预期:

```
Running bump tests...  
Bump tests run OK!  
cloud@server:~/gitWork/oscamp/arceos$
```

提示:

1. 可以参考现有的页分配器和字节分配器来实现相应的Trait。
2. 这个bump_allocator既是字节分配器，又是页分配器。所以必须同时实现BaseAllocator, ByteAllocator和PageAllocator三个Trait。这一点与现有的参考不同。